



Universidad Autónoma de Baja California

Facultad de Ciencias Químicas e Ingeniería

ISTE

Lenguajes Orientado Objetos

Unidad III
Herencia y Polimorfismo

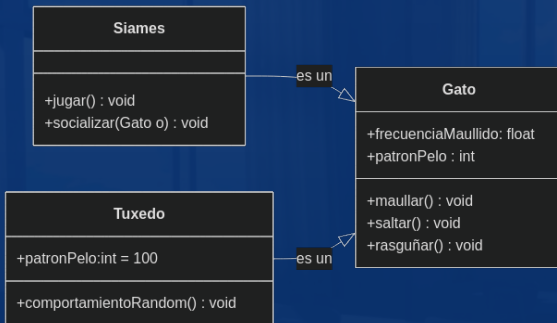
Es un concepto fundamental de la programación orientada a objetos la idea de herencia es que **puedes crear clases a partir de otras clases existentes**

Desde la perspectiva de implementación la herencia es cuando una clase adquiere de otra clase sus atributos y sus métodos **para reciclar código**.

En cuanto al diseño/modelado difiere un poco a la implementación , ya que se utiliza otra perspectiva en vez de modelar para reutilizar código se modela a partir de la **relación que tiene la clase o la esencia de la clase** por lo que decimos que **la clase B es una clase A**

Herencia

En **UML** la herencia se define con una flecha **triangular** sin rellenar, por ejemplo:



Una ventaja de herencia es que los métodos de **maullar**, **saltar** y **rasguñar** desde la perspectiva de implementación, se adquiere en las clases **Siameses** y **Tuxedo** así como los atributos de frecuenciaMaullido y patronPelo

Herencia Niveles de acceso

- **public:** se heredan a todos los hijos y tanto la clase como los objetos tienen acceso a estos.
- **private:** se heredan a las clase hijos **sin embargo las clases hijos no tienen acceso a estas.** (se ocupan getters y setters para acceder y modificar desde las clases hijos.)
- **protected:** se heredan a todos los hijos sin embargo actúan como variables privadas, solamente la clase padre y las clases hijas tiene acceso a estos

En Java a la clase padre se le conoce como **superclase** y representa a la clase de manera **general**

Y a las clases hijos se les conoce **subclases** y las puedes pensar **como clases especializadas de la superclase o como una extensión de la super clase.**

En java la herencia se define con la palabra reservada **extends** y el uso de la palabra reservada **super** (esta última hace referencia a la clase padre)

Sintaxis herencia

```
1 class sub_clase extends super_clase
2 {
3     subclase() //constructor
4     {
5         super(/*parámetros del constructor de la superclase*/);
6         //bloques de código
7     }
8 }
```



Del ejemplo anterior de Gato

Gato.java

```
1 class Gato{
2     public int patronPelo;
3     public float frecuenciaMaullido;
4     Gato(int patron, float maullido)
5     {
6         patronPelo = patron;
7         frecuenciaMaullido = maullido;
8     }
9     public void maullar()
10    {
11        System.out.println("Nyan!!");
12    }
13    public void saltar()
14    {
15        System.out.println("Gato:: Salta!!");
16    }
17    public void rasguñar()
18    {
19        System.out.println("Gato:: Rasguña!!");
20    }
21 }
```

Siames.java

```
1 | class Siames extends Gato
2 | {
3 |     Siames()
4 |     {
5 |         //se llama el constructor de Gato para inicializar los
6 |         // atributos que vienen de la clase Gato
7 |         super(5,1.5f);
8 |     }
9 |     public void jugar()
10 |    {
11 |        System.out.println("Siames:: Jugar!!");
12 |    }
13 |    public void socializar()
14 |    {
15 |        System.out.println("Siames:: Socializar!!");
16 |    }
17 | }
```



Tuxedo.java

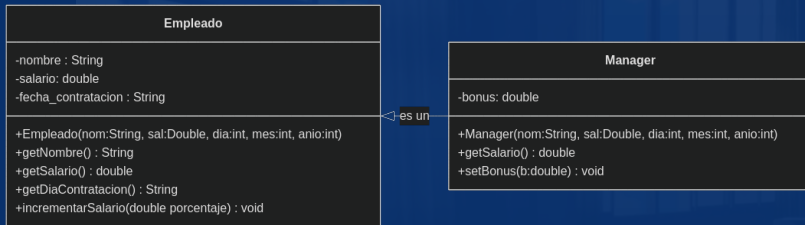
```
1 class Tuxedo extends Gato
2 {
3     Tuxedo()
4     {
5         //se llama el constructor de Gato para inicializar los
6         // atributos que vienen de la clase Gato
7         super(100,3.5f);
8     }
9     public void comportamientoRandom()
10    {
11        System.out.println("Tuxedo:: Comportamiento!!");
12    }
13 }
```

Demo.java

```
1 | class Demo
2 | {
3 |     public static void main(String[] args)
4 |     {
5 |         Siames siames = new Siames();
6 |         System.out.println("Siames pelo: "+siames.patronPelo);
7 |         System.out.println("Siames maullido: "+siames.frecuenciaMaullido);
8 |         Tuxedo tuxedo = new Tuxedo();
9 |         System.out.println("tuxedo pelo: "+tuxedo.patronPelo);
10 |        System.out.println("tuxedo maullido: "+siames.frecuenciaMaullido);
11 |
12 |        siames.maullar();
13 |        tuxedo.rasguñar();
14 |
15 |        siames.socializar(tuxedo);
16 |
17 |        tuxedo.comportamientoRandom();
18 |
19 |    }
20 | }
21 | }
```

Ejemplo #2

Por ejemplo en el caso de los roles de una empresa donde tenemos los empleados y el mánager.



Ejemplo #2

Empleado.java

```
1 | class Empleado
2 | {
3 |     private String nombre;
4 |     private double salario;
5 |     private String fecha_contratacion;
6 |     Empleado(String nom, double sal, int dia, int mes, int anio)
7 |     {
8 |         nombre = nom;
9 |         salario = sal;
10 |         fecha_contratacion = String.format("%d/%d/%d", dia, mes, anio);
11 |     }
12 |     public String getNombre()
13 |     {
14 |         return nombre;
15 |     }
16 |     public double getSalario()
17 |     {
18 |         return salario;
19 |     }
20 |     public String getDiaContratacion()
21 |     {
22 |         return fecha_contratacion;
23 |     }
24 |     public void incrementarSalario(double porcentaje)
25 |     {
26 |         double incremento = salario*porcentaje/100;
27 |         salario+=incremento;
28 |     }
29 | }
30 | }
```

Ejemplo #2

Manager.java

```
1 class Manager extends Empleado
2 {
3     private double bonus;
4     Manager(String nom, double sal, int dia, int mes, int anio)
5     {
6         super(nom, sal, dia, mes, anio);
7         bonus=0;
8     }
9     public double getSalario()
10    {
11        // en el caso como salario es privado
12        // es necesario usar la supeclase para acceder a el
13        // con la palabra reservada super que hace referencia a
14        // la superclase
15        double salario_base = super.getSalario();
16        return salario_base + bonus;
17    }
18    public void setBonus(double b)
19    {
20        bonus = b;
21    }
22 }
```

Ejemplo #2

Demo.java

```
1 | class Demo
2 | {
3 |     public static void main(String[] args)
4 |     {
5 |         Empleado[] empleados = new Empleado[3];
6 |         Manager man = new Manager("omar",123123,01,01,2001);
7 |         man.setBonus(5000);
8 |         //empleados[0].setBonus(5000) ERROR ya que el arreglo
9 |         //es de Empleado por lo que Empleado no tiene un método
10 |        //setBonus
11 |        empleados[0] = man;
12 |        empleados[2] = new Empleado("Mau",321321,02,05,2011);
13 |        empleados[3] = new Empleado("Raul",333111,11,11,2021);
14 |
15 |        for(var e: empleados)
16 |        {
17 |            System.out.println("nombre: "+e.getNombre());
18 |            // en el caso de manager se va ejecutar la implementación
19 |            // de getSalario de esa clase (polimorfismo)
20 |            // herencia nos permite modificar el comportamiento
21 |            // de los métodos de la superclase
22 |            System.out.println("salario: "+e.getSalario());
23 |        }
24 |    }
25 | }
```

Literalmente significa **muchas formas** esto significa que si usas herencia la misma **clase padre puede tomar diferentes formas ó lo métodos** que se heredan a las subclases pueden ejecutarse de diferentes formas dependiendo de la clase.

Por ejemplo: En el caso de la clase Manager se puede decir que **un Manager es un Empleado** por lo que Empleado puede tomar la forma de un Manager, lo contrario también se puede formular **No todo Empleado es un Manager.**

Haciendo referencia al acrónimo de Liskov Sustitution Principal de SOLID podemos decir que **se puede utilizar la subclase(clase hijo) en donde se necesite la superclase(clase padre)**

Por ejemplo cuando se puede usar las subclases en vez de la superclase

Demo

```
1 public class Demo
2 {
3     public static void main(String[] args)
4     {
5         Empleado[] arreglo = new Empleado[3];
6         arreglo[0] = new Manager("abc", 123123,01,01,01);
7         arreglo[0] = new Secretaria("uuus", 32313,21,11,01);
8         //etc
9     }
10 }
```

Aunque el arreglo es de la superclase, los elementos pueden ser de las subclases, al momento de ejecutarse tomarían los estados de la subclase.

Polimorfismo

Podemos usar polimorfismo para sobre escribir/agregar el comportamiento de una subclase a que sea independiente a la de la super clase.

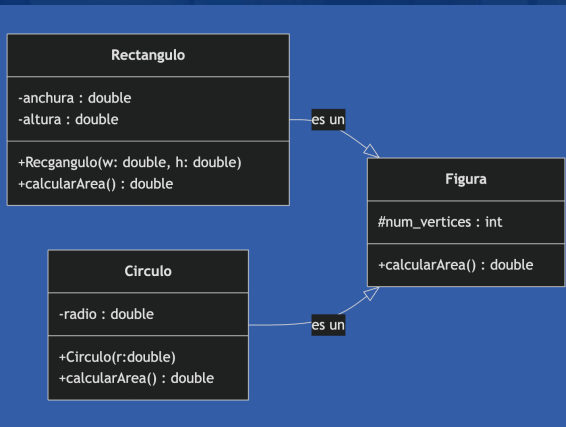


Figura.java

```
1 class Figura
2 {
3     Figura(int vert)
4     {
5         vertices = vert;
6     }
7     public double calcularArea()
8     {
9         //comportamiento de la superclase
10        return vertices;
11    }
12 }
```

Rectangulo.java

```
1 class Rectangulo extends Figura
2 {
3     Rectangulo(double w, double h)
4     {
5         super(4); //constructor padre
6         anchura = w;
7         altura = h;
8     }
9     public double calcularArea()
10    {
11        //comportamiento de la subclase
12        return altura*anchura;
13    }
14 }
```

Circulo.java

```
1 class Circulo extends Figura
2 {
3     Circulo(double r)
4     {
5         super(64);
6         radio = r;
7     }
8     public double calcularArea()
9     {
10         //comportamiento de la subclase
11         return Math.PI*radio*radio;
12     }
13 }
```

Demo.java

```
1 class Demo
2 {
3     public void main(String [] args)
4     {
5         //usando el principio de LSP
6         Figura[] figuras = new Figura[2];
7         figuras[0] = new Rectangulo(100.0,200.0);
8         figuras[1] = new Circulo(50.0);
9
10        for(var fig: figuras)
11        {
12            System.out.println(fig.calcularArea());
13        }
14
15    }
16 }
```

Prevenir la herencia

Hay veces que queremos que en nuestro árbol de herencia (jerárquico), unas clases no tiene sentido que tengan hijos, para limitar que las clases puedan tener herencia (ser superclases) entonces se utiliza la palabra reservada **final**

Por ejemplo si queremos que la clase ejecutivo no pueda tener subclases

Ejecutivo.java

```
1 public final class Ejecutivo extends Empleado
2 {
3     //código
4 }
```

También se puede hacer con los métodos, si queremos que algunos métodos no se pueda hacer polimorfismo (que la clase hija pueda sobre escribirla) entonces podemos agregar final

Ejecutivo.java

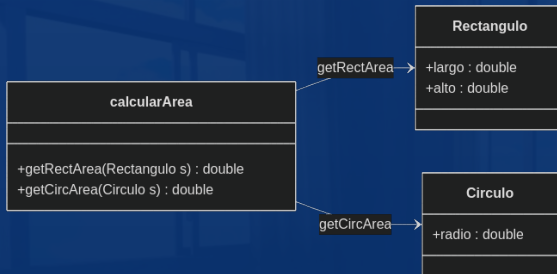
```
1 | class Empleado
2 | {
3 |     public final String getNombre()
4 |     {
5 |         return nombre;
6 |     }
7 | }
```

Este principio dice que **Clases deben estar abiertas a cambios pero cerradas a modificaciones**

En otras palabras, diseña/estructura tus clases tal forma que puedas crear nuevos comportamientos **sin modificar el código base**

Ejemplo: Open-Closed Principal

Supongamos que queremos hacer una calculadora de Áreas de figuras por lo que un diseño común es el siguiente:

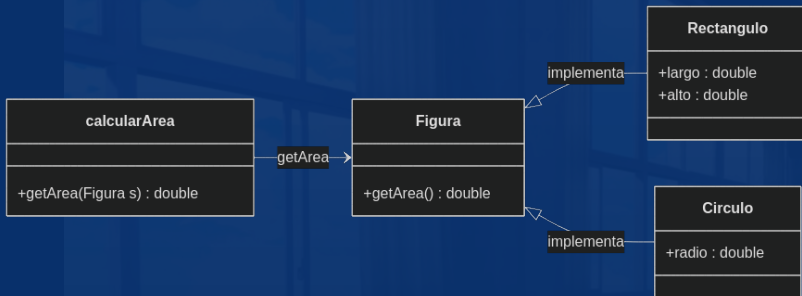


Ejemplo: Open-Closed Principal

Utilizando el principio, creamos una clase principal(padre) llamada figura con un método de calcularArea en donde los hijos (Rectángulo y Círculo) la implementarían dependiendo de sus necesidades.

De esta forma la clase calcularArea no necesita cambiar su código para agregar mas figuras, simplemente hace uso del método de la clase figura.

Ejemplo: Open-Closed Principal



Las clases derivadas pueden ser sustituidas por las clases bases mejor conocido como "Dynamic dispatching"

*No es buen diseño que la clase padre tenga todo los métodos**

Ejemplo Liskov Substitution Principal

En vez de tener un diseño así:

LSP

```
1 | class Demo
2 | {
3 |     public void main(String[] args)
4 |     {
5 |         Lista lista= new Lista();
6 |         lista.pushBack(new Carro("Honda",123123));
7 |         lista.pushBack(new Tren("Trupper",3212321));
8 |         lista.pushBack(new Barco("LaPinta",43421));
9 |
10 |        for(var p=lista.Inicio();! lista.yaTermino();p=lista.Next())
11 |        {
12 |            if(p instanceof Carro)
13 |            {
14 |                Carro c = (Carro)p;
15 |                c.Conducir();
16 |            }
17 |            else if(p instanceof Tren)
18 |            {
19 |                Tren t = (Tren)p;
20 |                t.Avanzar();
21 |            }
22 |            else
23 |            {
24 |                Barco b = (Barco)p;
25 |                b.Naufragar();
26 |            }
27 |        }
28 |    }
29 | }
```

Ejemplo Liskov Substitution Principal

podemos diseñar un método para manejar el vehiculo y que se implemente en las clases hijas. de esta forma podemos sustituir las clases hijas por la padre en el proceso.

LSP

```
1 class Demo
2 {
3     public void main(String[] args)
4     {
5         Lista lista= new Lista();
6         lista.pushBack(new Carro("Honda",123123));
7         lista.pushBack(new Tren("Trupper",3212321));
8         lista.pushBack(new Barco("LaPinta",43421));
9
10        for(var p=lista.Inicio();p!=null;lista.Current())
11        {
12            p.Manejar();
13        }
14    }
15 }
```